

EE-170L Computer Programming Lab

Laboratory Report — No. 11

Strings in C++

Student Name	Zawar Shah
Registration No.	BF25NWELE0677
Course	EE-170L – Computer Programming Lab
Lab No.	11
Topic	C-Strings, string Object, String Functions
Semester	Spring 2026
Institution	UET Peshawar

1. Lab Objectives

- Understand the two types of strings in C++: C-style char arrays and string class objects.
- Declare, initialize, and display C-strings using char arrays with null terminator.
- Use `cin`, `cin.get()`, and `getline()` appropriately for reading strings including spaces.
- Declare and use string objects from the C++ Standard Library.
- Apply built-in string functions: `strlen()`, `strcpy()`, `strcat()`, `strcmp()`, `strupr()`, `strlwr()`.
- Solve practical string tasks involving length calculation and concatenation.

2. Theoretical Background

2.1 C++ Strings — Overview

A string is a collection of characters. C++ supports two types of strings:

- **C-Strings (C-style):** Character arrays of type `char` terminated with the null character `\0`.
- **string Objects:** Objects of the `string` class from the C++ Standard Library. No fixed length, dynamically extensible.

2.2 C-String: Declaration and Initialization

A C-string is a char array where the last element is always the null character `\0`. The compiler automatically appends `\0` when a string literal is assigned:

Examples:

```
char myString[] = "Hello";           // auto adds \0 - holds 6 chars
char myString[6] = "Hello";          // explicit size with space for \0
char myString[] = {'H','e','l','l','o','\0'}; // manual init
char myString[100] = "Hello";        // extra space is fine - not all used
```

- The string "Hello" has 5 visible characters plus 1 null terminator = 6 bytes minimum.
- Unlike arrays, it is not necessary to use all allocated space in a char array.
- The null character `\0` marks where the string ends in memory.

2.3 Reading Strings — cin vs cin.get() vs getline()

The method used to read a string determines whether spaces are captured:

Using cin (stops at whitespace):

```
#include <iostream>
using namespace std;
int main()
{
    char str[100];                // declare char array to hold input
    cout << "Enter a string: ";
    cin >> str;                  // reads only until first space
    cout << "You entered: " << str << endl;
    return 0;
}
```

Note:

- If the user types "Electrical Engineering", cin stores only "Electrical" — the space acts as a terminator.

Using cin.get() (reads spaces, C-string version):

```
#include <iostream>
using namespace std;
int main()
{
    char str[100];                // buffer to hold the full line
    cout << "Enter a string: ";
    cin.get(str, 100);           // reads up to 99 chars including spaces
    cout << "You entered: " << str << endl;
    return 0;
}
```

2.4 string Object and getline()

The string class provides a dynamic string with no fixed length. Use getline() to read a full line including spaces:

```
#include <iostream>
using namespace std;
int main()
{
    string str;                  // declare a string object (no size
    // needed)
    cout << "Enter a string: ";
    getline(cin, str);           // reads entire line including spaces
    cout << "You entered: " << str << endl;
    return 0;
}
```

Output:

```
Enter a string: University of Engineering and Technology
You entered: University of Engineering and Technology
```

2.5 Built-in String Functions

C++ provides a set of functions in <cstring> for manipulating null-terminated C-strings:

Function	Syntax	Description
strcpy()	strcpy(s1, s2);	Copies string s2 into s1

strcat()	strcat(s1, s2);	Appends s2 to the end of s1
strlen()	strlen(s);	Returns the number of characters in s (excluding \0)
strcmp()	strcmp(s1, s2);	0 if equal; <0 if s1<s2; >0 if s1>s2
strrev()	strrev(s);	Reverses string s in place
strlwr()	strlwr(s);	Converts all characters to lowercase
strupr()	strupr(s);	Converts all characters to uppercase

3. Example Programs

3.1 Read and Display a Single-Word String Using cin

```
#include <iostream>
using namespace std;
int main()
{
    const int MAX = 80;           // maximum characters allowed in string
    char str[MAX];               // declare char array to store input
    cout << "Enter a string: ";
    cin >> str;                  // read one word (stops at whitespace)
    cout << "You entered: " << str << endl; // display the stored string
    return 0;
}
```

Output:

```
Enter a string: Welcome
You entered: Welcome
```

3.2 Read a Full Line Using cin.get()

```
#include <iostream>
using namespace std;
int main()
{
    char str[100];               // buffer large enough for the full line
    cout << "Enter a string: ";
    cin.get(str, 100);          // reads up to 99 chars – captures spaces
    cout << "You entered: " << str << endl;
    return 0;
}
```

Output:

```
Enter a string: Electrical Engineering
You entered: Electrical Engineering
```

4. Lab Tasks & Solutions

Task 1 — String Length Calculation

Declare and initialize a string variable 'message' with a phrase of your choice. Use strlen() to calculate and display its length.

```
#include <iostream>
#include <cstring>           // required for strlen() and other C-string
                              functions
using namespace std;
int main()
{
    // Declare and initialize a C-string with a phrase
    char message[] = "EE-170L Computer Programming Lab";

    // Calculate the length of the string (excluding the null terminator \0)
    int len = strlen(message);

    // Display the original string and its computed length
    cout << "String : " << message << endl;
    cout << "Length : " << len << " characters" << endl;

    return 0;
}
```

Output:

```
String : EE-170L Computer Programming Lab
Length : 32 characters
```

How it works:

- `strlen()` counts characters from index 0 up to (but not including) the null terminator `\0`.
- The result is an integer representing the number of visible characters in the string.
- The string literal "EE-170L Computer Programming Lab" has 32 characters plus 1 null byte = 33 bytes total in memory.

Task 2 — String Concatenation

Declare two string variables 'firstName' and 'lastName'. Concatenate them using `strcat()` to form a full name and display the result.

```
#include <iostream>
#include <cstring>           // required for strcat() and strcpy()
using namespace std;
int main()
{
    // Declare first name – use a buffer large enough to hold the full name
    // after concatenation
    char firstName[50] = "Zawar";

    // Declare last name as a separate C-string
    char lastName[] = " Shah"; // leading space separates first and last
    name

    // Display both names before concatenation
    cout << "First Name : " << firstName << endl;
    cout << "Last Name : " << lastName << endl;

    // Concatenate lastName onto the end of firstName
    // strcat() appends lastName to firstName and updates firstName in place
    strcat(firstName, lastName);

    // Display the combined full name
    cout << "Full Name : " << firstName << endl;

    return 0;
}
```

Output:

```
First Name : Zawar
Last Name  : Shah
Full Name  : Zawar Shah
```

How it works:

- `firstName` must be declared with a buffer large enough to hold both strings after concatenation.
- `strcat()` finds the `\0` at the end of `firstName` and begins writing `lastName` from that position.
- A leading space in " Shah" ensures the full name is properly separated.
- After `strcat()`, `firstName` contains the complete full name "Zawar Shah".

5. C-String vs string Object — Key Differences

Feature	C-String (char array)	string Object
Type	char array	string class object
Size	Fixed at declaration	Dynamic, grows as needed
Null terminator	Required (<code>\0</code>)	Managed automatically
Read with spaces	<code>cin.get()</code>	<code>getline()</code>
Concatenation	<code>strcat()</code>	<code>+</code> operator or <code>append()</code>
Length	<code>strlen()</code>	<code>.length()</code> or <code>.size()</code>
Header needed	<code><cstring></code>	<code><string></code> (or auto-included)

6. Conclusion

This lab introduced the two string types in C++ and the built-in functions that support string manipulation. Key takeaways:

- **C-Strings:** Char arrays terminated with `\0`. Fixed size; require `<cstring>` functions for manipulation.
- **string Objects:** Flexible, dynamically sized strings from the Standard Library. Simpler syntax with operators like `+`.
- **Input Methods:** `cin` stops at spaces; `cin.get()` and `getline()` capture full lines including whitespace.
- **`strlen()`:** Returns the number of visible characters, excluding the null terminator.
- **`strcat()`:** Appends one string to another; the destination buffer must be large enough to hold the combined result.

String handling is essential for real-world C++ programs. Mastery of both C-strings and string objects prepares you for file I/O, user input validation, and data parsing in upcoming labs.